CSE 471 – System Analysis and Design

Short Question & Answer Notes

Lectures 1, 2 & 3 | BRAC University

LECTURE 1: Introduction and SDLC

Q. What is System Analysis and Design?

Ans: System Analysis and Design is the process of studying a business problem and finding the best way to build an information system to solve it. It involves analyzing what the organization needs, designing the system, building it, and delivering it to users. The system analyst is the key person who handles this entire process.

Q. What is the SDLC (Systems Development Life Cycle)?

Ans: SDLC is the step-by-step process used to develop an information system. It has four main phases: Planning (why build it?), Analysis (what should it do?), Design (how will it work?), and Implementation (build and deliver it). Each phase produces deliverables used in the next phase.

Q. What is an Information System (IS)?

Ans: An Information System is a software application that manages large amounts of data and shares information across an organization. It has five main components: Hardware, Software, Databases, Networks, and Procedures. Together these components collect, process, and provide information.

Q. What are the five components of an Information System?

Ans: The five components are: (1) Hardware – physical devices like monitor, keyboard, processor; (2) Software – programs that process data; (3) Databases – collections of related data; (4) Networks – systems that connect computers; and (5) Procedures – instructions that combine all components to produce output.

Q. Who is a Systems Analyst and what is their role?

Ans: A Systems Analyst is a professional who bridges the gap between business needs and technology. They analyze business problems, identify areas for improvement, and design information systems to fix those problems. They work with all team members and serve as change agents in the organization.

Q. What skills does a Systems Analyst need?

Ans: A systems analyst needs three key types of skills: (1) Technical skills – understanding existing technology and new system tools; (2) Business skills – knowing how IT can help business processes; and (3) Analytical/Problem-solving skills – to identify and resolve problems at both the project and organization level.

Q. Why is Systems Analysis important? Why do so many IT projects fail?

Ans: Systems Analysis is critical because most software errors (45%) occur during requirements and design phases, and fixing them after delivery can cost 100 times more. Studies show 30% of large IT projects are cancelled, and 50% go over budget. Proper analysis helps avoid these costly failures from the very beginning.

Q. What are the four phases of the SDLC?

Ans: The four phases are: (1) Planning – identify business value, analyze feasibility, develop work plan; (2) Analysis – understand requirements and model the new system; (3) Design – decide how the system will be built and design all components; (4) Implementation – build, test, install, and support the system.

Q. What is a Methodology in system development?

Ans: A methodology is a formalized, step-by-step approach for implementing the SDLC. It provides a structured way to develop systems. Common methodology categories include process-centered, data-centered, and object-oriented. Examples include Waterfall, RAD, Agile, Prototyping, and DevOps.

Q. What is the Waterfall Development methodology?

Ans: Waterfall is a traditional methodology where development flows downward through phases — Planning, Analysis, Design, and Implementation — one at a time. Each phase must be fully completed before the next begins. It is simple and easy to manage but is inflexible to change once a phase is done.

Q. What is Rapid Application Development (RAD)?

Ans: RAD is a methodology that focuses on building systems quickly by using prototypes and getting frequent user feedback. Instead of following every step slowly, developers build a working version fast, show it to users, get feedback, and improve it. It reduces development time and increases user satisfaction.

Q. What is Agile Development?

Ans: Agile is a flexible software development methodology that works in small iterations called 'sprints'. Teams deliver working pieces of the system regularly (every 2–4 weeks), get user feedback, and adjust accordingly. It is great for projects where requirements keep changing and quick delivery is needed.

Q. What is DevOps?

Ans: DevOps is a methodology that combines software Development and IT Operations into one continuous process. It removes the barrier between developers (who write code) and operations teams (who run servers). The goal is to release software faster, more reliably, and with fewer errors using automation.

Q. Why was DevOps created? What problem does it solve?

Ans: Before DevOps, the development team and operations team worked separately, which caused delays, miscommunication, and human errors. Code was deployed manually and testing happened late. DevOps solved this by making both teams work together, automating processes, and using containers to share code smoothly.

Q. What are the steps of DevOps?

Ans: DevOps follows 8 steps: Plan → Code → Build → Test → Release → Deploy → Operate → Monitor. These steps form a continuous loop. The idea is that everything should be continuous — Continuous Development, Continuous Testing, Continuous Integration, Continuous Deployment, and Continuous Monitoring.

Q. What are the advantages of DevOps?

Ans: DevOps offers several advantages: reduced chance of product failure, faster time to market, improved flexibility, better team efficiency, and a clearer product vision. Since development and operations work together, issues are caught early and software is delivered more quickly and reliably.

Q. What are the challenges of DevOps?

Ans: The main challenges of DevOps include: difficulties with integration of tools and teams, setting up automated testing, relatively high initial costs, choosing the right toolset from many options, and finding skilled professionals who understand both development and operations.

Q. What are the five Information Systems team roles?

Ans: The five major IS roles are: (1) Business Analyst – focuses on business process improvements; (2) Systems Analyst – designs the IS solution; (3) Infrastructure Analyst – handles technical/hardware needs; (4) Change Management Analyst – manages people and organizational change; (5) Project Manager – oversees the whole project.

LECTURE 2: Feasibility Analysis

Q. What is a System Request?

Ans: A System Request is a formal document prepared by the project sponsor that describes why a new system should be built. It includes the business need, business requirements, expected business value, and any special issues or constraints. The approval committee reviews it to decide whether to proceed with the project.

Q. What are the key elements of a System Request?

Ans: A System Request has five key elements: (1) Project Sponsor – the person initiating the project; (2) Business Need – the reason for the system; (3) Business Requirements – what the system must do; (4) Business Value – the benefits it will bring; and (5) Special Issues or Constraints – deadlines, security needs, or limitations.

Q. What is Business Value, and what are its types?

Ans: Business value is the benefit that a new system will create for the organization. It has two types: (1) Tangible value – benefits that can be clearly measured, like a 5% increase in sales or lower operating costs; (2) Intangible value – benefits that are hard to measure, like improved customer service or a better competitive position.

Q. What is Feasibility Analysis?

Ans: Feasibility Analysis is the process of determining whether a proposed system is worth building. It checks if the system is technically possible, economically beneficial, and organizationally acceptable. It also helps identify major risks associated with the project before committing resources.

Q. What are the three types of feasibility?

Ans: The three types are: (1) Technical Feasibility – can we actually build it with available technology?; (2) Economic Feasibility – will the benefits outweigh the costs (should we build it)?; and (3) Organizational Feasibility – will users accept and use it (if we build it, will they come?).

Q. What is Technical Feasibility?

Ans: Technical feasibility checks whether the organization has the technical ability to build the system. It considers: familiarity with the application and business domain, familiarity with the required technology, the size of the project (people, time, features), and how easily the new system can be integrated with existing technology.

Q. What is Economic Feasibility?

Ans: Economic feasibility determines whether the financial benefits of the system outweigh its costs. It involves performing a cost-benefit analysis where you identify all costs (development + operational) and all benefits (tangible + intangible), assign dollar values, and calculate metrics like ROI, NPV, and break-even point.

Q. What are Development Costs and Operational Costs?

Ans: Development costs are one-time costs to build the system, such as hardware purchases, software licenses, and labor costs for programmers. Operational costs are ongoing costs to run the system after it is built, such as maintenance, operational labor, and software subscriptions. Both must be included in the cost-benefit analysis.

Q. What is Return on Investment (ROI)?

Ans: ROI measures how much money you earn back compared to how much you invested. A high ROI means the project is financially worth it. It can be calculated per year or for the entire project period. ROI is desirable when the total benefits received are greater than the total costs spent on the project.

Q. What is the Break-Even Point?

Ans: The Break-Even Point is the point in time when the total benefits earned equal the total costs spent. Before this point, the project is losing money; after it, the project becomes profitable. A longer break-even time means higher risk. It can be shown visually with a graph plotting cumulative costs and benefits over time.

Q. What is Present Value (PV) and why is it used?

Ans: Present Value is the current value of money that will be received in the future. Money received in the future is worth less than money today because of interest rates and inflation. We use PV to compare future benefits and costs fairly. Formula: PV = Cash Flow / (1 + interest rate)^n, where n is the number of years.

Q. What is Net Present Value (NPV)?

Ans: NPV is the difference between the total present value of all benefits and the total present value of all costs. If NPV is positive, the project is financially worthwhile. If NPV is negative, the project costs more than it earns. NPV helps organizations make smarter investment decisions by accounting for the time value of money.

Q. What is Organizational Feasibility?

Ans: Organizational feasibility checks whether the system will actually be accepted and used by the people in the organization. It is assessed in two ways: (1) Strategic Alignment – does the project match the organization's overall goals and strategy?; (2) Stakeholder Analysis – will the key people (users, managers, sponsors) support the system?

Q. What is Stakeholder Analysis?

Ans: Stakeholder Analysis identifies all people and groups who are affected by or can affect the new system. It considers three groups: Project Champions (people who strongly support the system), Organizational Management (who authorize and fund it), and System Users (who will actually use it daily). Their support is key to project success.

Q. Who is a Project Sponsor?

Ans: A Project Sponsor is a senior person in the organization who initiates and champions the proposed system. They recognize the business need, understand the expected value, and want the project to succeed. They prepare the system request document and serve as the primary business-side contact throughout the project.

LECTURE 3: Requirements Determination

Q. What is a Requirement in system development?

Ans: A requirement is a statement that describes what a system must do or what characteristics it must have. Requirements come from different perspectives: what the business needs, what users want to do, what the software should perform, what quality the system should have, and how the system should be built.

Q. What are the five types of requirements?

Ans: The five types are: (1) Business Requirements – what the organization needs overall; (2) User Requirements – what users need to accomplish their tasks; (3) Functional Requirements – what specific functions the software must perform; (4) Nonfunctional Requirements – quality characteristics like speed and security; (5) System Requirements – how the system should be built technically.

Q. What are Functional Requirements?

Ans: Functional requirements define the specific tasks or processes that the system must perform to support users. For example, 'the system must allow users to search for products' or 'the system must generate monthly sales reports'. They describe what the system does, not how it does it.

Q. What are Non-Functional Requirements?

Ans: Non-functional requirements describe the quality and behavioral properties that the system must have. They include: (1) Operational – what environment the system runs in; (2) Performance – speed, capacity, reliability; (3) Security – who can access what; and (4) Cultural and Political – legal, cultural, or company-specific rules the system must follow.

Q. What is the difference between Functional and Non-Functional Requirements?

Ans: Functional requirements describe what the system does (its features and functions), like 'the system must send a confirmation email'. Non-functional requirements describe how well the system performs, like 'the system must respond within 2 seconds' or 'the system must be available 24/7'. Both are equally important for system success.

Q. What are the five main Requirement Gathering Techniques?

Ans: The five most commonly used techniques are: (1) Interviews – talking directly with users and stakeholders; (2) JAD Sessions – group workshops for collaborative requirements gathering; (3) Questionnaires – written surveys for large groups; (4) Document Analysis – reviewing existing documents and reports; and (5) Observation – watching how users currently work.

Q. What is Problem Analysis?

Ans: Problem Analysis is a technique where analysts ask users to identify problems with the current system and suggest solutions. It is simple and quick, but improvements are usually small and incremental. It rarely leads to big improvements with high business value because users tend to suggest minor fixes rather than major redesigns.

Q. What is Root Cause Analysis?

Ans: Root Cause Analysis goes deeper than Problem Analysis. Instead of asking users for solutions, analysts ask them to list their problems and all possible root causes behind those problems. The analysts then find solutions that address the root causes, especially ones that are common to multiple problems — leading to more impactful improvements.

Q. What is Duration Analysis?

Ans: Duration Analysis measures how long each step of a process takes and compares it to the total time the entire process takes. A big gap between the two means the process is badly fragmented and inefficient. Solutions include process integration (fewer people with broader roles) or parallelization (doing steps at the same time).

Q. What is Activity-Based Costing?

Ans: Activity-Based Costing calculates the cost of each individual step in a business process, considering both direct and indirect costs. By finding the most expensive steps, analysts know where to focus their improvement efforts to save the most money. It helps prioritize where redesigning the process will have the greatest financial impact.

Q. What is Benchmarking in requirements analysis?

Ans: Benchmarking means studying how other organizations perform the same business process so you can compare and learn from them. There are two types: formal (structured research into competitors) and informal (experiencing another business's process as a customer). It helps identify best practices that can be adopted to improve your own system.

Q. What is Technology Analysis?

Ans: Technology Analysis is a technique where analysts and managers together create a list of important and interesting technologies and then explore how each technology could be applied to improve the business. The group discusses potential benefits. It helps organizations discover innovative ways to use new technology they may not have considered.

Q. What is Activity Elimination?

Ans: Activity Elimination asks: 'What would happen if we simply removed this step from the process?' By using a 'force-fit' approach — imagining that each activity is eliminated — analysts discover which steps are truly necessary and which are wasteful or redundant. This can lead to a much simpler and more efficient process design.

Q. What is JAD (Joint Application Design)?

Ans: JAD is a structured group workshop where business users, managers, and analysts come together to gather requirements collaboratively. Instead of interviewing people one by one, everyone meets in a session to discuss needs, agree on requirements, and resolve conflicts together. JAD saves time and produces more complete and agreed-upon requirements.

Q. What is the goal of the Analysis Phase in SDLC?

Ans: The main goal of the Analysis Phase is to fully understand what the new system must do and create a detailed requirements definition. This phase takes the general ideas from the system request and develops them into functional models, structural models, and behavioral models. The final output is the system proposal submitted for approval.

CONCEPTUAL QUESTIONS

Conceptual – Lecture 1: Introduction & SDLC

Q. Why do large IT projects fail more often than small ones? Explain with real-world context.

Ans: Large IT projects involve more people, longer timelines, and complex requirements — all of which increase the chance of miscommunication and bad decisions. The Standish Group found that only 2% of projects exceeding $100 million in labor costs succeed. Real examples include Bank of America losing over $1 billion on a failed accounting system, and the Ariane 5 rocket crash caused by a software reuse error. These failures show that without proper systems analysis upfront, even well-funded projects can collapse.

Q. A company wants to build a new system but skips the Analysis phase to save time. What could go wrong?

Ans: Skipping analysis is extremely risky. About 45% of all software errors occur during requirements and design — and fixing them after delivery costs up to 100 times more than catching them early. Without proper analysis, the team may build the wrong system, miss key user needs, or design a system that doesn't fit business goals. The result is often an over-budget, late, or completely abandoned project — exactly what happened in many real-world IT failures.

Q. How does a Systems Analyst act as a 'change agent' in an organization?

Ans: A change agent is someone who drives change in how an organization works. A systems analyst does this by identifying outdated or inefficient processes, designing new information systems to replace them, and then helping people adapt to the change. They don't just build software — they challenge the current way things are done and train and motivate users to accept and use new systems effectively. This role requires both technical expertise and people skills.

Q. Compare Waterfall and Agile methodologies. When would you choose one over the other?

Ans: Waterfall is a linear approach — each phase (Planning, Analysis, Design, Implementation) must be completed before the next begins. It works best when requirements are clear and unlikely to change, like a government payroll system. Agile works in short sprints and adapts to changing requirements — it's ideal for startups or products where user feedback shapes the final product, like a mobile app. If the project scope is fixed, use Waterfall; if flexibility and speed matter, use Agile.

Q. Why is DevOps considered an evolution beyond Agile? What gap does it fill?

Ans: Agile improved how software is developed in short sprints, but it still left a gap: the handoff between the development team (who build code) and the operations team (who deploy and run it). This handoff caused delays, errors, and blame-shifting. DevOps fills this gap by merging both teams into one continuous workflow using automation and containers. While Agile focuses on building software fast, DevOps ensures that software is also deployed, operated, and monitored seamlessly and continuously.

Q. If you were a Project Manager, how would you decide which SDLC methodology to use for a new project?

Ans: The choice depends on several factors: (1) Are requirements clear and stable? → Use Waterfall. (2) Do requirements change frequently? → Use Agile. (3) Do you need a quick working prototype? → Use RAD or Prototyping. (4) Do development and deployment need to be continuous and automated? → Use DevOps. (5) Is the project very large with many teams? → Structured or Parallel Development may be better. A good PM evaluates team size, project risk, timeline, and user involvement before deciding.

Conceptual – Lecture 2: Feasibility Analysis

Q. Why is it important to perform feasibility analysis before starting a project? What happens if it is skipped?

Ans: Feasibility analysis ensures the organization doesn't commit time and money to a project that is technically impossible, financially unwise, or one that users will reject. Skipping it can lead to wasted investment — like the Bank of America case, where $60 million was spent trying to fix a broken system before it was abandoned entirely. Feasibility analysis acts as an early warning system, identifying risks before they become expensive failures.

Q. A project passes technical and economic feasibility but fails after launch. What might have gone wrong?

Ans: This is a classic failure of Organizational Feasibility — the 'if we build it, will they come?' question. Even a technically sound and profitable system will fail if employees refuse to use it, if it doesn't align with company strategy, or if key stakeholders don't support it. This is why stakeholder analysis and strategic alignment are critical. A system that employees see as a threat to their jobs, or that management doesn't champion, will be ignored or sabotaged regardless of how good it technically is.

Q. Why do we use Net Present Value (NPV) instead of just comparing raw costs and benefits?

Ans: Money has a time value — $1,000 today is worth more than $1,000 three years from now because of interest rates and inflation. If we simply add up raw costs and benefits across years without adjustment, we overestimate future benefits. NPV solves this by converting all future cash flows to their value in today's money using the formula PV = Cash Flow / (1 + rate)^n. A positive NPV means the project truly creates value after accounting for the cost of waiting for those returns.

Q. What is the difference between tangible and intangible business value? Why are both important?

Ans: Tangible value is easy to measure in dollars — for example, 'this system will reduce operating costs by 10%' or 'increase sales by $500,000'. Intangible value is real but hard to quantify — such as improved customer satisfaction, stronger brand reputation, or better employee morale. Both matter because a system that only looks good on paper (tangible) but frustrates users (intangible loss) will ultimately fail. Organizations must consider both types when deciding whether a system is truly worth building.

Q. How does the Break-Even Point help a decision-maker assess risk in a project?

Ans: The break-even point tells you how many years it will take before the project starts making a profit — i.e., when cumulative benefits equal cumulative costs. A short break-even period means lower risk, because the organization recovers its investment quickly. A long break-even period is risky because many things can go wrong over time: budgets may be cut, the technology may become outdated, or business needs may change. Decision-makers use this alongside NPV and ROI to evaluate whether the wait is worth it.

Q. Can a project be economically feasible but technically infeasible? Give an example.

Ans: Yes, absolutely. Imagine a hospital wants to build an AI-powered diagnostic system that would save millions in misdiagnosis costs (economically feasible), but their IT team has no expertise in machine learning, their servers can't handle the computational load, and the system can't integrate with their existing patient records software (technically infeasible). In this case, the financial case is strong, but the organization simply doesn't have the technical capability or infrastructure to build and run it reliably.

Conceptual – Lecture 3: Requirements Determination

Q. Why is gathering the 'right' requirements considered the hardest part of system development?

Ans: Requirements are hard to get right because users often don't know exactly what they want until they see it. There are also communication gaps between business users and technical analysts, vague or conflicting requests, and the challenge of capturing both functional needs (what the system does) and non-functional needs (how well it does it). A classic illustration is the 'tree swing' cartoon — what the customer described, what the analyst understood, and what was finally built were all completely different things.

Q. Why might interviewing alone not be enough to gather complete requirements?

Ans: Interviews are great for getting detailed opinions, but they are one-on-one and time-consuming. A single interview may miss important perspectives from other stakeholders. Users may also struggle to articulate needs they take for granted — things they do automatically every day. That's why analysts combine interviews with other techniques: observation (to see what users actually do, not just what they say), document analysis (to understand existing processes), questionnaires (to reach many people), and JAD sessions (to resolve conflicts collaboratively).

Q. What is the difference between Problem Analysis and Root Cause Analysis? Which is more valuable and why?

Ans: Problem Analysis asks users to describe problems and suggest solutions — it is quick but only produces surface-level improvements. Root Cause Analysis goes deeper: it identifies why problems are happening (the root causes) rather than just what the problems are. By finding common root causes across multiple problems, analysts can design solutions that fix several issues at once, creating far more business value. Root Cause Analysis is generally more valuable because it leads to systemic, impactful improvements rather than small patches.

Q. How do functional and non-functional requirements work together to make a successful system?

Ans: Think of it this way: functional requirements define what the system does (the features), while non-functional requirements define how well it does it (the quality). A banking app might functionally allow money transfers (functional), but if it takes 30 seconds per transaction and crashes under load (non-functional failure), users will abandon it. A system that does the right things but does them slowly, insecurely, or unreliably will still fail. Both types must be captured, prioritized, and satisfied together for the system to truly succeed.

Q. When would you use Duration Analysis over Activity-Based Costing, and vice versa?

Ans: Use Duration Analysis when the main problem is that a process takes too long — it reveals time inefficiencies caused by fragmentation (too many handoffs between people or departments). Use Activity-Based Costing when the main concern is that a process is too expensive — it pinpoints which steps consume the most money. In practice, both can be used together: first find the most expensive steps (costing), then see if those steps are also taking too long (duration). Together they give a complete picture of where to redesign the process.

Q. A company is choosing between questionnaires and JAD sessions for requirements gathering. What factors should guide this decision?

Ans: Choose Questionnaires when: the user group is very large and spread across locations, responses need to be anonymous, or questions are straightforward yes/no or rating-type questions. Choose JAD Sessions when: requirements are complex and need group discussion, stakeholders have conflicting views that need to be resolved together, or speed of consensus matters. JAD is richer and more interactive but requires all key people in the same room. Questionnaires are efficient for breadth; JAD is better for depth and agreement.